

Recommending Test Cases in Agile Software Development with a Taxonomy-Guided KNN Approach

Ednaldo Dilozenzo
Federal Institute of Paraíba, PB, Brazil
ednaldo.dilozenzo@ifpb.edu.br

Felipe Ramos
Federal Institute of Paraíba, PB, Brazil
felipe.ramos@ifpb.edu.br

Danyllo Albuquerque
VIRTUS/UFCG
Federal University of Campina
Grande, PB, Brazil
danyllo.albuquerque@virtus.ufcg.edu.br

Mirko Perkusich
VIRTUS/UFCG
Federal University of Campina
Grande, PB, Brazil
mirko@virtus.ufcg.edu.br

Hyggo Almeida
VIRTUS/UFCG
Federal University of Campina
Grande, PB, Brazil
hyggo@virtus.ufcg.edu.br

Angelo Perkusich
VIRTUS/UFCG
Federal University of Campina
Grande, PB, Brazil
perkusic@virtus.ufcg.edu.br

Abstract

Context: Agile testing faces unique challenges, including short cycles, evolving requirements, and the constant need to update test suites. Reusing validated test cases can improve coverage and efficiency, but it is often hindered by fragmented documentation and evolving repositories. **Objective:** This study introduces a Recommender System (RecSys) to support test case reuse in agile contexts by exploiting historical user stories and a domain-specific taxonomy. **Method:** We implemented a K-Nearest Neighbors (KNN)-based RecSys that retrieves user stories similar to a target and recommends their associated test cases. The approach emphasizes transparent and interpretable recommendations. We assessed it through a multi-method design: (i) an offline experiment with 217 user stories and 1,077 test cases from two companies to identify the best configuration using recall, precision, and F-measure; and (ii) an online case study in two agile projects to evaluate its real-world impact. **Results:** In the offline phase, the best configuration reached 67.5% recall, 37.8% precision, and an F_2 score of 51%. In the online pilot, recall remained similarly high at 65.1%, while precision rose substantially to 63.1%, alongside a 38.8% expansion of the test suite. **Conclusion:** These findings show that our taxonomy-based RecSys effectively supports test reuse in agile settings, yielding consistent results both offline and in real-world settings. Its transparent logic, low infrastructure needs, and alignment with agile workflows make it a lightweight and practical alternative to more complex reuse solutions.

CCS Concepts

• **Software and its engineering** → *Software creation and management*.

Keywords

Test Case Reuse, Agile Testing, Recommender System, User Stories, k-Nearest Neighbors

ACM Reference Format:

Ednaldo Dilozenzo, Felipe Ramos, Danyllo Albuquerque, Mirko Perkusich, Hyggo Almeida, and Angelo Perkusich. 2026. Recommending Test Cases in

Agile Software Development with a Taxonomy-Guided KNN Approach. In *The 41st ACM/SIGAPP Symposium on Applied Computing (SAC '26)*, March 23–27, 2026, Thessaloniki, Greece. ACM, New York, NY, USA, Article 4, 9 pages. <https://doi.org/10.1145/3748522.3779891>

1 Introduction

Over the past two decades, Agile Software Development (ASD) has emerged as the dominant paradigm for delivering software in short, iterative cycles [25, 29, 47]. This shift emphasizes speed, responsiveness to change, and waste minimization, creating new demands on software testing. Test strategies must adapt accordingly—becoming faster, more flexible, and more integrated into development workflows. Yet, developers and testers often spend disproportionate time navigating complex information spaces, searching for reusable artifacts instead of focusing on value-adding work [39].

Reusing validated test cases has long been seen as a promising way to improve coverage and efficiency without incurring additional effort [20]. As organizations accumulate substantial testing knowledge, systematic reuse could reduce redundancy and accelerate validation. However, realizing this potential in practice remains challenging. Industrial repositories typically suffer from inconsistent documentation, weak structure, and rapid evolution [41], all of which hinder the retrieval of relevant cases during sprint planning or user story development.

Several strategies have been proposed to support test reuse via Recommender Systems (RecSys). Collaborative filtering has shown that structurally similar requirements can guide retrieval [9], and signature-based matching has been applied for cross-project reuse [45]. Other approaches rely on tag-driven retrieval [6] or similarity across code and documentation [46]. While effective in specific contexts, these solutions often rely on heavyweight infrastructure—such as semantic models, embeddings, or deep learning pipelines—and tend to operate at the code level or in regression testing, rather than supporting proactive reuse at the user story level.

More recently, Large Language Models (LLMs) have expanded possibilities, enabling automated test case generation [21, 27, 48] and traceability recovery [23, 53]. These tools deliver strong generative performance, but raise concerns about reproducibility, interpretability, and alignment with domain-specific agile practices. Moreover, they typically aim to replace rather than reuse validated assets, which limits trust and sustainability in industrial projects.

Taken together, these observations reveal a clear gap: agile teams still lack a lightweight, transparent, and early-stage mechanism for



This work is licensed under a Creative Commons Attribution 4.0 International License. SAC '26, Thessaloniki, Greece
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2294-3/2026/03
<https://doi.org/10.1145/3748522.3779891>

reusing test cases directly at the user story level—where decisions about coverage and validation begin. To address this gap, we propose a taxonomy-guided Recommender System tailored for agile contexts. Our approach leverages structured user stories and accumulated test knowledge to support reuse in early sprint planning. It builds on a lightweight k -Nearest Neighbors (KNN) algorithm, which offers transparent and interpretable similarity judgments. Unlike prior solutions, this design emphasizes interpretability and low adoption cost, making it suitable for teams seeking actionable support without heavy infrastructure.

This work makes three main contributions. First, it introduces a taxonomy-based RecSys for agile test reuse, built around user story classification. Second, it empirically evaluates the system through a two-phase design, combining controlled offline experiments with an in-situ industry deployment to contrast recall-precision trade-offs. Third, it distills actionable insights for both researchers and practitioners, demonstrating how taxonomy-guided reuse can enhance test coverage, strike a balance between exploratory recall and practical precision, and minimize redundant effort.

The remainder of the paper is organized as follows: Section 2 reviews related work; Section 3 presents the proposed approach; Section 4 describes the evaluation design; Section 5 reports and interprets the findings; Section 7 outlines threats to validity; and Section 8 concludes with implications and future directions.

2 Background and Related Work

RecSys for SE aims to reduce developers' effort by surfacing relevant artifacts for specific tasks [39]. They have primarily focused on debugging, implementation, and maintenance, with an emphasis on code reuse and visualization. Yet, most remain reactive, code-centric, and dependent on heavyweight infrastructure. Gasparic et al. [18] highlighted gaps in adaptability and the narrow focus on code-level reuse, stressing the potential for proactive recommenders in other SE areas.

Our work builds on this by exploring a rarely addressed reuse opportunity: test cases linked to semantically similar user stories in agile projects. In the following, we present the assumptions and prior research that guide our design.

Requirements Similarity as a Proxy for Reuse. We assume that textual similarity between requirements implies similarity in downstream artifacts (e.g., code and tests). Thus, our RecSys adopts a hybrid content–collaborative approach. Abbas et al. [2] empirically confirmed a positive correlation between requirement similarity and the similarity of their associated components, also noting the role of contextual factors such as domain knowledge and traceability. This supports our assumption that semantically close user stories are likely to share reusable test cases.

Bera et al. [9] proposed a structured model for documenting agile requirements. Although they did not implement a RecSys, their results showed that well-structured and annotated requirements improve retrieval. Broader work in requirements traceability [23, 53] similarly demonstrates that metadata such as purpose or stakeholder role enhance reuse. Together, these studies justify using structured user story similarity as a proxy for test case reuse.

To operationalize this idea, we adopt the taxonomy proposed by Dilozenzo et al. [14], which is derived from 25 Web Information Systems (WIS) projects. It classifies stories by feature type in a consistent (Module, Operation) structure, with three main modules: *Authentication* (e.g., login, password recovery, permissions), *Registration* (CRUD operations), and *Management* (dashboards, reports, notifications). The corresponding operations are summarized in

Table 1. Each story can be labeled with one or more pairs, enabling multi-label classification and richer comparisons. While we use this taxonomy for its empirical basis, our approach is not tied to it; any structured labeling scheme that enables meaningful comparison across requirements can support reuse and be adapted to other domains.

Table 1: WIS taxonomy.

Module	Operations
Authentication	Perform login with username and password; Perform login with OAuth; Password recovery; First login; Validate user permissions; Update user profile
Registration	Data retrieval; Update data; Insert data; Remove data; Change data
Management	Visualize Dashboard; Export PDF report; Export XLS report; Notify through application; Notify through e-mail

The modules reflect typical WIS functions: *Registration* involves CRUD (i.e., Create, Read, Update, Delete) operations; *Authentication* handles system access and control; and *Management* concerns operational data, such as dashboards and reports. Although we use this taxonomy due to its empirical basis, our method is not tied to it. Any structured labeling scheme that enables meaningful comparison across requirements, especially with respect to the type of feature involved, can support our approach. This flexibility allows adaptation to other domains or evolving taxonomies.

Test Case Recommendation. Early work on test case reuse explored semantic models and collaborative filtering. Landhäuser et al. [28] populated an ontology with user stories and test steps, applying ontology matching for retrieval. Despite a high recall (77.5%), precision was low (8.68%), and the ontology was costly to maintain. Jianxin et al. [19] later applied item-based collaborative filtering with a k -NN metric, reporting lower error than baselines but limited industrial generalizability and high computational cost.

Another line emphasizes code-level similarity. Reza et al. [46] proposed BDTTCR, which balances similarity across code, comments, and control-flow graphs, outperforming prior methods on a corpus of 13,000 tests. Sun et al. [45] introduced MuTCR, a multi-level signature matcher for cross-project reuse, which improved accuracy and ran three times faster than baselines. However, these methods operate at method level and depend on static analysis—constraints that limit their applicability during early agile planning.

Collaborative filtering has also supported prioritization of test suites. Azizi and Do [7] reordered tests in CI pipelines using textual tags and failure data, while Yang et al. [52] combined collaborative filtering with a knowledge graph of defect chains. Such solutions require mature projects with rich defect histories, focusing on prioritization rather than early reuse.

More recently, LLMs have shifted attention from reuse to generation. Surveys, such as those by Shao et al. [48], review over 100 studies. Empirical evaluations show that models like GPT-4 and Gemini can generate high-coverage unit tests [21, 27]. Yet, these black-box approaches raise concerns about reproducibility and traceability, and do not leverage validated assets within organizations.

Overall, most existing RecSys (i) operate at low-level code granularity, (ii) rely on heavy infrastructure, or (iii) focus on regression prioritization rather than proactive reuse. Our work follows a distinct path: we classify user stories using a validated taxonomy and

apply a transparent similarity metric to retrieve reusable test cases before coding begins. While requiring structured labeling, the approach is lightweight and practical where historical user stories exist but advanced tools or large corpora are lacking.

3 Proposed Approach

To support agile teams in reusing test cases, we developed a RecSys-based approach grounded in a hybrid content-collaborative paradigm. The system receives a target requirement and retrieves similar user stories to recommend associated test cases from past or ongoing projects. Test cases, in general, are requirement-driven and validate system behavior under controlled inputs [10]. Thus, if requirements are similar, their test cases are also likely to be similar, as shown by Abbas et al. [2]. Our focus is ASD, where User Stories are the main carriers of product knowledge [49]. However, they are lightweight artifacts, designed for communication rather than persistence, which hinders traceability and reuse.

To address this, we enriched User Stories with additional attributes to clarify their purpose and enable similarity inference [13, 15, 36, 37]. Specifically, we adopted the WIS taxonomy of Dilorenzo et al. [14], which classifies requirements in (Module, Operation) pairs. For example, user stories of type (*Authentication*, *First login*) are considered similar even with different textual descriptions.

Our solution also incorporates 59 templates of Acceptance Criteria (AC), derived from 200 real-world examples, categorized as either functional or non-functional. A functional requirement (FR), for instance, specifies: “When submitting <entity> data, it must be persisted in the database.” In addition, we standardized non-functional requirements (NFRs) using the model of Ramos et al. [36], which categorizes each NFR by type, attribute, and sentence. For example, a “Security” NFR may include the attribute “Authentication” and the rule: “The system must validate user credentials before granting access.”¹

We formalized our problem by adapting the general definition of recommendation problems presented by Adomavicius and Tuzhilin [3] to our context: Let U be the set of all User Stories and T the set of all test cases that can be recommended. Let f be a utility function that measures the usefulness of a test case t to a user story u , i.e.,

$$f : U \times T \rightarrow \mathbb{R},$$

where higher values indicate greater usefulness. For each user story $u \in U$, we wish to recommend the test case $t'_u \in T$ that maximizes f . Formally, the problem is expressed in Equation 1 (adapted from [3]).

$$\forall u \in U, t'_u = \arg \max_{t \in T} f(u, t). \quad (1)$$

Here, t'_u is the (possibly multi-element) set of test cases that attain the maximum utility for the user story u .

Our RecSys is composed of three components (Figure 1). The *data collector* searches structured information from User Stories together with project data to build *User Story profiles*. These profiles are then processed by the *data transformer*, which converts them into characteristic vectors that allow the calculation of similarity between the retrieved User Stories and the one under development, referred to as the *target User Story*. Finally, the *recommender* analyzes the vectors, computes the similarity between the target and retrieved User Stories, and suggests test cases associated with the most similar stories, ranking them by relevance.

¹More details are available in the supplementary material.

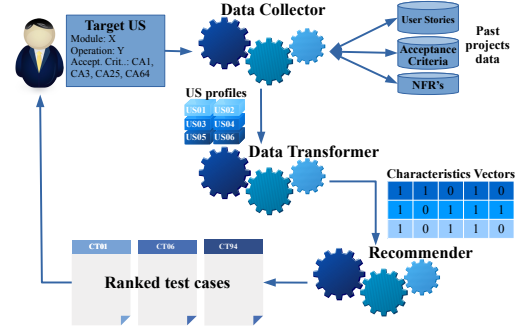


Figure 1: Components of the Test cases RecSys.

The recommendation process starts with the creation of a new User Story, referred to as the *Target User Story*. The *data collector* then retrieves historical information from past projects in the form of a *US profile*, which includes US data, ACs, NFRs, domain, and platform, and forwards it to the *data transformer*. For this to work, User Stories must contain attributes for classification, becoming what we call *Semi-structured User Stories*.

Table 2 illustrates examples of US profiles retrieved by the *data collector*. Both ACs and NFRs are multi-valued attributes, meaning that each User Story may have several associated criteria. NFRs, however, are not always directly linked to a single story; they may come from project-level data, the definition of done, or specific User Stories that define them [35]. Thus, we treat such NFRs as associated with all stories in a project.

The RecSys passes the data retrieved by the *data collector*, as well as the target User Story (u_t), to the *data transformer*. Then, it compares them to generate an encoded matrix of characteristic vectors. For this, the *data transformer* uses the *one-hot encoding* technique [11]. Figure 2 shows the matrix, considering u_7 from Table 2 as the target User Story ($u_t = u_7$), for demonstration purposes. As can be seen, the resulting matrix has almost the same fields as the User Story profile. Unlike, it has one column for each AC and NFR from the target User Story.

	Module	Oper.	Plat.	Domain	AC1	AC2	NFR1	NFR2
u_t	1	1	1	1	1	1	1	1
u'_1	1	0	1	0	0	0	1	1
u'_2	1	0	1	0	0	0	1	1
u'_3	1	0	1	0	0	0	1	1
u'_4	0	0	1	0	0	0	1	1
u'_5	1	1	1	1	1	1	1	0
u'_6	1	0	1	0	1	1	1	1

Figure 2: Example of a matrix of characteristic vectors.

The *Recommender* receives the matrix of characteristic vectors, applies collaborative filtering based on user attributes [22], and recommends test cases from the nearest neighbors of the target User Story. Similarity is computed using the KNN algorithm [34], which measures the distance between the encoded target User Story (u_t) and each retrieved story (u'_i). These distances are normalized to the interval $[0, 1]$ [30]. Finally, the K User Stories with the highest

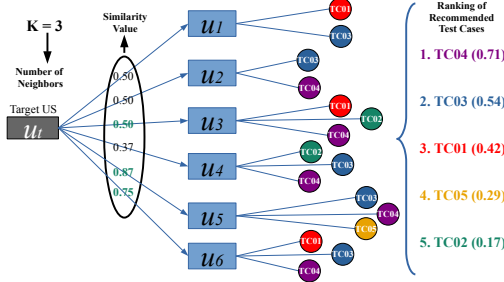
Table 2: Examples of User Story profiles extracted from projects' developed User Stories.

ID	Module	Operation	Platform	Domain	AC's	NFR's
u_1	Registration	change_inserted_data	Web	Support	8 9 10 11 12 13 14	1 2
u_2	Registration	insert_data	Web	Factory	1 2 3 6 43	1 2
u_3	Registration	retrieve_data	Web	Factory	15 17 20 21 57	1 2
u_4	Authentication	execute_login_with_username_and_password	Web	Factory	25	1 2
u_5	Registration	delete_data	Web	Management	49 50 51	1
u_6	Registration	insert_data	Web	Finance	25 49 50 51	1 2
u_7	Registration	delete_data	Web	Management	49 50	1 2

similarity scores are selected, and their associated test cases are recommended to the RecSys user.

$$\text{sim}(u_t, u') = 1 - \frac{\sum_{i=1}^n |u_{ti} - u'_{ti}|}{n}. \quad (2)$$

Figure 3 illustrates a hypothetical execution of the *recommender*. With $K = 3$, the most similar User Stories from Table 2 (u_3 , u_5 , and u_6) are selected, with scores of 0.50, 0.87, and 0.75, respectively. Their associated test cases are then ranked using Equation 3, which averages the similarity scores of all neighbors linked to a given test case. Equation 4 shows the calculation for $TC04$. Note that ratings serve only illustrative purposes and do not affect the recommendations themselves.

**Figure 3: Calculation of K Nearest Neighbors to test case recommendation considering the heuristic.**

$$r(tc_j) = \frac{\sum_{i=1}^n \text{sim}(u_t, u'_i)}{K}, \quad (3)$$

$$r(tc_{04}) = \frac{0.50 + 0.87 + 0.75}{3} = 0.71. \quad (4)$$

More than one User Story may share the same similarity score. For example, Figure 3 shows u_1 , u_2 , and u_3 with similarity 0.50, where the standard algorithm selected u_1 simply because it was retrieved first. To address this, we introduced a heuristic that, in case of ties, prioritizes the User Story with the largest number of associated test cases.

4 Methodology

We validated the proposed RecSys through a multi-method design that combined an offline experiment with historical data and an *in-situ* pilot in industry projects. This approach enabled us to evaluate

both the algorithm's performance under controlled conditions and its practical utility in real agile teams.

The *offline phase* came first, as recommended in RecSys research [16], since it is cheaper, easier to replicate, and suitable for tuning. Using labeled user stories and their associated test cases, we tested several configurations to identify the most effective in terms of recall, precision, and F-measure. After identifying the best-performing configuration, we proceeded to the *online phase*, where we deployed the RecSys in two ongoing software projects at an industrial partner. This pilot case study aimed to assess whether the recommendations supported testers in reusing validated test cases and enhancing their test suites during actual sprint planning and development.

We did not compare our system with other recommenders (e.g., Bera et al. [9], Ge and Liu [19]), since these rely on fundamentally different infrastructures—such as structured requirement models, collaborative filtering, or knowledge graphs. Our approach depends on a domain-specific taxonomy and semantic labeling of user stories, making direct comparisons non-trivial. Moreover, no benchmark dataset exists for a fair evaluation. Therefore, we focused on internal effectiveness across configurations and external utility in industrial deployment.

Offline Validation. We evaluated our RecSys with a *10-fold cross validation*, a common strategy in RecSys research [40]. This approach partitions data into K folds, using $K - 1$ for training and one for testing, reducing bias and avoiding overspecialization.

Training Dataset. To mitigate the *Cold-Start* problem [43], we collected data from two companies executing Scrum projects. The dataset comprised 217 User Stories from 13 projects (covering *Mobile*, *Web*, and *Desktop*), 530 Acceptance Criteria, and 1077 distinct test cases. All User Stories were manually classified according to the WIS taxonomy (Section 2.1), and test cases received standard identifiers. Classifications were reviewed and disagreements resolved by discussion.

Experiment Design. Our goal was to identify the configuration with the best performance in recommending test cases. We tested combinations of distance metrics and number of neighbors ($K = 1-10$), also comparing results with and without the heuristic for tie-breaking, yielding 60 configurations. The Research Question (RQ_{off}) was: *Which RecSys configuration achieves the best results for test case recommendation?* Independent variables were K , distance algorithms (Cosine [3], Euclidean [38], Manhattan [38], Jaccard [5], Canberra, and Chebyshev [26]), and the heuristic. The dependent variable was recommendation performance.

We assess the recommender with *precision*, *recall*, and their balance through the weighted F -measure (F_β). Since recall is more critical in our context, we set $\beta = 2$. A test case recommender acts as an *exploratory assistant*: during sprint planning, testers prefer to

review all potentially relevant cases and discard irrelevancies rather than miss a critical one. Prior work echoes this view—false positives can be filtered, but false negatives may compromise quality [44]. Thus, F_2 emphasizes recall while still penalizing an overload of irrelevant suggestions (see Equation 5).

$$F\text{-measure}(\beta) = \frac{(\beta^2 + 1) \cdot \text{precision} \cdot \text{recall}}{(\beta^2 \cdot \text{precision}) + \text{recall}} \quad (5)$$

Before running the validation, we observed that *Canberra*, *Euclidean*, *Jaccard*, and *Manhattan* produced identical results, since characteristic vectors contain only binary values (0/1), yielding equivalent distances [4]. Therefore, to simplify the analysis, we retained only *Euclidean*.

With this setup, we executed the cross-validation and analyzed the data to answer RQ_{off} and determine the most effective configuration for test case recommendation. To do so, we first applied a *Multiple Comparison Test*, which is appropriate for assessing differences among several groups without a control [17]. Once significant differences were confirmed, we then used a *Friedman post hoc* analysis to rank the configurations and identify the best performer in terms of $F\text{-Measure}(\beta)$.

Online Validation. After identifying the best RecSys configuration in the offline phase, we conducted a pilot to evaluate its ability to support Scrum teams in reusing test cases and strengthening their test suites. For this purpose, we conducted a descriptive case study to gather evidence of RecSys’s impact on real projects. Accordingly, we defined the main RQ_{on} : *How does the RecSys affect test case development in agile projects?*. To clarify this objective, we further decomposed the RQ as follows:

- RQ_{on1} : To what extent does the test cases recommender increase the test suite completeness for agile projects?
- RQ_{on2} : To what extent does the test cases recommender enable the test cases reuse for agile projects?

In RQ_{on1} we examined whether combining recommendations with testers’ own cases produced a more complete test suite than the original, while in RQ_{on2} we investigated whether the RecSys anticipated cases that testers would naturally develop, thus enabling reuse. To address these questions, we analyzed User Stories and test cases from two projects of the same organization—six sprints from one and one sprint from the other. All stories were classified according to the taxonomy described in Section 3. At the end of each sprint, the RecSys was executed with the sprint’s User Stories as input, and its recommendations were compared with the test cases created by the testers. Finally, the results were discussed with the teams, and the cases were classified as follows:

- *Developed*: the test cases created by the project’s testers and not recommended by the RecSys.
- *Reuse*: the test cases that testers created and were also recommended by the RecSys.
- *Accepted*: the test cases that the RecSys recommended and the testers considered eligible for being incorporated in the project’s test suite, but have not created them.
- *Rejected*: the test cases recommended by the RecSys, not developed by testers, and not considered relevant by them.

For test cases classified as *Accepted*, we also assessed their relevance to the project. Added cases were categorized as *high* (critical to the project), *medium* (intermediate relevance), or *low* (added value but no impact if omitted). Based on this classification, we

defined three metrics: *test suite increase percent* (ts_inc_perc , Eq. 6); *reuse percent*, i.e., the proportion of recommended cases actually reused by testers ($reuse_perc$, Eq. 7); and *reuse precision*, i.e., the accuracy of recommendations ($reuse_precision$, Eq. 8).

$$ts_inc_perc = \frac{\#Accepted}{\#Developed + \#Reuse} \quad (6)$$

$$reuse_perc = \frac{\#Reuse}{\#Developed + \#Reuse} \quad (7)$$

$$reuse_precision = \frac{\#Accepted + \#Reuse}{\#Accepted + \#Reuse + \#Rejected} \quad (8)$$

5 Results and Discussion

This section reports the findings from our two-phase evaluation. We first present the offline validation, where multiple configurations of the recommender were tested using standard information retrieval metrics (Section 5.1). Next, we describe the online pilot in two agile projects, assessing its impact on test suite completeness and test case reuse (Section 5.2). Finally, we integrate both perspectives, comparing offline and online outcomes, and discussing implications for research and practice (Section 5.3).

Offline Validation. The results of the $F\text{-measure}(2)$ analysis are summarized in Figure 4. As shown, the values are higher when using a smaller number of neighbors. This occurs because precision is considerably greater with fewer neighbors, while recall remains relatively stable.

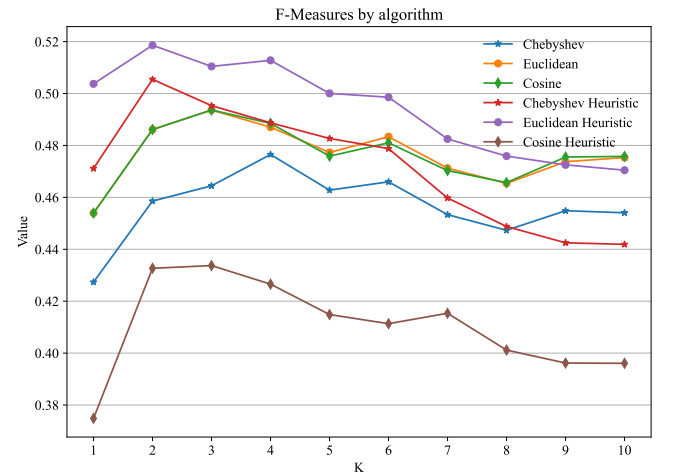


Figure 4: F-Measure(2) for each value of K.

Statistical Analysis. After cross-validation, we analyzed the data to answer the RQ in Section 4 and determine the best configuration for recommending test cases. For this, we applied a *Multiple Comparison Test*, suitable for comparing several groups of independent variables without a control [17]. As a preliminary step, we assessed the normality of the data distribution [12], since this guides the subsequent statistical procedures. To do so, we performed a hypothesis (H) test with the following assumption:

- H_0 : The data population is normally distributed.
- H_1 : The data population is not normally distributed.

Several methods can test data normality, but Razali et al. [33] showed that *Shapiro-Wilk* performs best across distributions and sample sizes. We therefore applied it with a 5% significance level ($\alpha = 0.05$). For all 60 configurations, the test returned values below α , allowing rejection of the null hypothesis (H_0) and indicating that the data are not normally distributed.

Given this result, we employed a non-parametric test to determine whether the independent variables affected performance. Among such methods, García et al. [17] found *Friedman* to be most effective for comparisons with more than five groups. Applying Friedman with $\alpha = 0.05$ yielded a p -value below 0.05, confirming statistically significant differences among configurations. To rank them and identify the best performer in terms of F -Measure(2), we then applied a *Friedman post hoc* analysis. Table 3 presents the ten top-performing configurations (Algorithm, K , and Heuristic).

Table 3: Friedman test configurations ranking.

Rank	Algorithm	K	Heuristic	Groups
1	Euclidean	3	Yes	a
2	Euclidean	2	Yes	ab
3	Cosine	3	No	ab
4	Euclidean	3	No	ab
5	Chebyshev	2	Yes	abc
6	Chebyshev	3	Yes	abcd

The configurations were ranked alphabetically: group “a” contains the best, followed by “b”, and so on. To answer RQ_{off} and select a configuration for the online experiment, we chose one from group “a”—specifically, the *Euclidean* metric with $K = 3$ and the neighbor-selection heuristic—since it had the highest mean in the dataset, although any option in group “a” would be statistically valid. This configuration achieved 67.5% recall, 37.8% precision, and an F -measure(2) of 51%. Compared to Landhäuser et al. [28], our results show higher precision but lower recall, likely due to differences in the level of granularity addressed by each study.

Answer to RQ_{off}

The configuration *Euclidean distance*, $K = 3$, *heuristic = yes* outperformed all 59 alternatives ($p < 0.05$, Nemenyi post hoc). It achieved **67.5% recall**, **37.8% precision**, and an F_2 score of **0.510**. This setting is therefore carried forward to the online deployment.

Online Validation. Table 4 presents a list of recommended test cases for a sample US (i.e., *project registration* for an agile tool under development), classified according to their comparison with those developed by the company’s team. The result shows that one test case could be reused (Reuse), two test cases were included in the company’s test suite (Accepted), and two test cases were not included in the test suite (Rejected).

Table 4: Test cases analysis for US project registration.

Test Case	Status
Successful project registration	Reuse
Cancel project registration	Accepted
Register project with existing id	Rejected
Verify access by roles	Rejected
Close windows before executing	Accepted

Table 5 reports the metrics defined in Section 4. The RecSys increased the test suite by 38.79%, indicating that many recommended cases had not been considered by testers. It also covered 65.09% of the cases eventually developed, meaning that testers would only need to create the remaining 34.91%. In theory, this suggests a reduction in testing effort, although measuring this effect was beyond our scope. Additionally, we analyzed the relevance of the accepted test cases, based on testers’ perception, to assess the tool’s efficiency. The results showed that 47.5% of the accepted cases were considered of low relevance, 27.1% of medium relevance, and 25.4% of high relevance. This distribution suggests that, although many reused cases add value, only a portion are critical to the project.

Table 5: Metrics defined to evaluate pilot project and results adjusted.

Metric	Result
Test suite increase percent (ts_inc_perc)	38,79%
Reuse percent (reuse_perc)	65,09%
Precision (reuse_precision)	63.09%
Rejection percent (reject_perc)	36.91%

Among the rejected test cases (36.91%), the main reason was Business compliance (Table 6). This occurred because some User Stories included business rules absent in similar stories from the company’s database. A common case was entity editing: while certain stories allowed changing an entity’s ID, others considered similar did not.

Table 6: Summary of rejected test cases.

Reason	Result
Business compliance	85.81%
Other requirement	0.70%
Not applicable	7.80%
Too simple	4.96%

Therefore, we analyzed the data to answer the study’s RQs. For RQ_{on1} , we examined whether the RecSys increases test suite completeness. Based on the *ts_inc_perc* metric, we found evidence that testers do not cover all relevant cases on their own, since the growth in suite size came from recommended cases judged important for product quality.

Most of the accepted cases were of low relevance, but a considerable portion was classified as medium or high. Even less critical cases can add value when incorporated, and the inclusion of high-priority ones—though fewer in number—can significantly improve final product quality. Overall, the RecSys helped build more complete test suites than those produced solely by testers.

For RQ_{on2} , we investigated whether the RecSys enables test case reuse. Based on the *reuse_perc* metric, we observed that more than half of the test cases could have been reused rather than developed if the tool had been applied during execution. It is important to note that reuse does not translate directly into equivalent effort reduction, since testers must still adapt cases to specific business rules. Nonetheless, this effort is mitigated by the observed precision of 63.09%, indicating that testers could select and reuse most of the recommended cases.

Answer to RQ_{on1} (suite completeness)

Recommendations accepted by testers increased the test suite by **38.8 %** (ts_inc_perc), with 35 % of these additions rated medium–high relevance.

Answer to RQ_{on2} (reuse enablement)

The RecSys reproduced **65.1%** of the test cases that testers eventually wrote, at a recommendation precision of **63.1 %**. Hence, two-thirds of manually authored tests could have been reused with minor adaptation effort.

Integrated interpretation of results. This part synthesizes findings from both offline and online phases to assess the consistency, practical value, and limitations of the proposed RecSys. By comparing controlled experiment outcomes with real-world results, we highlight the effectiveness, areas for improvement, and broader applicability of this approach.

Alignment between offline and online results. The configuration selected offline—Euclidean distance with $K = 3$ and heuristic tie-breaking—achieved 67.5% recall, 37.8% precision, and an F_2 score of 0.51 in 10-fold cross-validation. In practice, it reproduced 65.1% of the test cases ultimately developed (a recall drop of only 2.4 points) and delivered a higher actionable precision of 63.1%. This consistency suggests the configuration generalizes well under real-world constraints.

Impact on test reuse and suite completeness. Across 41 User Stories, the RecSys recommended 334 test cases. Of these, 132 matched tests were already written (reused), and 79 were accepted as new additions, increasing the test suite by 38.8%.² Overall, about 63% of recommendations were useful, showing the system can both support reuse and extend coverage with little overhead.

Limitations and failure cases. The main reason for rejection (86% of cases) was incompatibility with implicit or project-specific business rules not captured by the taxonomy. This highlights a key improvement opportunity: enriching user stories with semantic tags or rule-level metadata to minimize false positives.

Applicability and generalization. While our study focused on WIS, the approach primarily relies on the availability of a domain-specific taxonomy. Substituting this with domain-tailored alternatives requires minimal adaptation. These findings suggest that lightweight, taxonomy-guided k -NN recommenders can effectively promote early-stage test reuse in agile projects without requiring complex infrastructure or large-scale datasets.

6 Main Implications

Our two-phase study shows that a lightweight, taxonomy-guided KNN recommender can (i) in the offline evaluation, achieve high recall but moderate precision (67.5% recall, 37.8% precision), and (ii) in the online evaluation, maintain similarly high recall (65.1%) while substantially improving precision to 63.1%, leading to a 38.8% expansion of test suites. These findings highlight the trade-off between exploratory recall in controlled settings and actionable precision in practice, with important implications for both research and industry.

Implications for research. Our findings reveal new opportunities and design challenges for RecSys in software testing: (i) *Automated story-level labeling is the next enabling step.* Our approach depends on classifying user stories with a domain taxonomy—a task performed manually in this study. This highlights an

open challenge: adapting recent LLM-based requirement classifiers (e.g., NoBERT [24], PRcBERT [32], FNReq-Net [42], LLM-based [8]) to user-story corpora so that feature vectors can be built with minimal human effort; (ii) *Hybrid similarity models* should also be explored for cold-start scenarios. Early iterations of a project may contain few labeled stories, limiting the quality of the neighbors. Code-level LLM embeddings, as used in API and traceability recommendation [31, 50], could provide approximate neighbors until the taxonomy corpus grows, improving early recall without manual labels; (iii) *Retrieve-then-generate pipelines* could further boost coverage. In our online pilot, 38.8% of the final test-suite growth came from test cases absent in the reuse pool but accepted after recommendation. This gap suggests an opportunity to pair retrieval with LLM-based test generation, delivering a single ranked list that reuses validated tests first and proposes new ones when reuse is impossible; Finally, (iv) *human-centric evaluation* remains under-studied. Offline precision of 37.8% implies testers must triage false positives. Future work should run controlled user studies to measure cognitive load, trust in recommendations, and the time savings (or overhead) associated with triage—data that complement traditional IR metrics and guide interface design.

Implications for Practice. For agile teams aiming to “shift-left” testing, the study offers actionable guidance: (i) *Tag recurring story types first to realize early benefits.* Our pilot showed a 38.8% increase in test-suite size after introducing taxonomy-based reuse. Teams that start by tagging the story categories that recur every sprint may realize similar early gains, even before the entire backlog is classified; (ii) *Introduce a lightweight triage routine to handle false positives.* About 37% of recommended tests were rejected, and 86% of those stemmed from undocumented business rules. A brief sprint-level review—e.g., adding rule keywords to acceptance criteria and skimming the suggestion list—could reduce this noise and maintain precision near the 63% observed in our pilot; (iii) *Continuously curate the historical corpus.* Recommendation quality depends on up-to-date links between user stories and tests. Automating the ingestion of new artifacts and re-indexing similarity vectors (e.g., nightly) prevents drift and sustains precision and recall over time; Finally, (iv) *communicate reuse in terms of business value.* While precision and recall metrics resonate with QA engineers, decision-makers often look for cost and quality indicators. Teams should map reuse statistics onto approximate effort saved, cycle-time reduction, or defect-containment potential when reporting value to stakeholders.

In summary, our results demonstrate tangible benefits of lightweight test-case recommendation but also highlight open research challenges — automated labeling, hybrid similarity, and human-in-the-loop evaluation — and practical considerations such as triage discipline and corpus hygiene that must be addressed for widespread industrial impact.

7 Threats to Validity

This section discusses potential threats to validity, following the categories of Wohlin et al. [51].

Conclusion validity. Although the offline dataset comprises 217 user stories and 1,077 test cases, some taxonomy categories (e.g., *Authentication*) are underrepresented. When a category appeared fewer than K times in a fold, the RecSys could not retrieve neighbors. We mitigated this by discarding such folds and reporting effective sample size, but the reduced counts lower statistical power. Moreover, testing 60 algorithm configurations raises the risk of Type-I error; to counter this, we applied Friedman with post hoc Nemenyi at $\alpha = 0.05$, though borderline differences remain

²79 additions over 203 originally authored cases.

uncertain. Data normality was rejected by Shapiro–Wilk, justifying non-parametric analysis, but other assumptions (e.g., independence of folds) may not hold, since cross-validation folds can correlate if stories share many test cases. Future work could apply nested cross-validation to reduce this risk.

Internal validity. User stories were manually mapped to taxonomy categories by the first author and double-checked by two co-authors, with 6% disagreement resolved by discussion. While this limits error, latent bias may persist, and inter-rater reliability was not measured; future work should involve independent raters and report kappa statistics. In the online study, the Scrum team utilized a plug-in that duplicated some tests; we excluded auto-generated duplicates from reuse counts, although residual confounding is still possible. Finally, tester proficiency may have improved across sprints, which in turn may have positively affected reuse. As the pilot covered only seven sprints (41 stories) and no time-series analysis was performed, maturation effects cannot be ruled out.

Construct validity. We relied on precision, recall, and F -measure ($\beta = 2$) for offline evaluation, assuming they correlate with tester utility. These metrics, however, overlook the effort to triage false positives or adapt reused tests. Complementary measures (e.g., Mean Reciprocal Rank, time-to-acceptance) could provide richer insight. For viability, we evaluated only KNN with six distance functions; other RecSys families (e.g., matrix factorization, BERT/LLM embeddings) were not explored, limiting construct coverage. Finally, similarity was computed on binary taxonomy vectors: semantically close stories in different categories may be undervalued, while dissimilar ones in the same category may be overvalued. Textual embeddings or hierarchical weights could mitigate this threat.

External validity. Training data came from two Brazilian vendors; the online experiment was conducted at one of them, and both focused on WIS. Results may differ in safety-critical domains or in contexts with richer/poorer test documentation. Our dataset (1,077 tests) is modest compared with large-enterprise repositories; scaling may require ANN indexing to reduce KNN's $O(nK)$ cost. The approach also presumes (i) user stories conform to the taxonomy, (ii) each story links to at least $K = 3$ historical tests, and (iii) similar wording/semantics persist across projects. Without these, recall may drop or cold-start issues emerge. Finally, both pilot teams emphasized automation and shared test ownership; reuse may be lower in cultures with siloed testers and developers.

8 Final Remarks

This paper introduced a RecSys based on a hybrid content–collaborative approach to support the reuse of test cases in ASD. We empirically configured and evaluated the RecSys through a two-phase study. In the offline evaluation, using 217 user stories and 1,077 test cases collected from 13 projects across two companies, the best-performing setup—Euclidean distance, $K = 3$, and a heuristic for tie-breaking—achieved high recall but moderate precision: 67.5% recall, 37.8% precision, and an F_2 score of 0.510. This confirmed the suitability of the approach for exploratory reuse, where missing cases are costlier than inspecting false positives. In the online evaluation, performance remained consistent, with recall remaining similarly high (65.1% of the test cases that teams eventually authored were reproduced), while precision rose substantially to 63.1%. This precision gain indicates that recommendations were much closer to testers' actual needs in real projects. Beyond aligning with authored cases, the RecSys also expanded the test suites

by 38.8%, surfacing relevant test cases that would have otherwise been missed.

Altogether, these results show that the system generalizes well from controlled experiments to industrial use, improving precision in real agile settings. For research, the findings indicate the need to automate story-level labeling with LLMs, explore hybrid similarity models to mitigate cold-start issues, combine reuse with test generation to expand coverage, and conduct human-centric evaluations to assess trust and effort savings beyond IR metrics. For practice, they suggest tagging recurring story types early to realize reuse benefits, adopting lightweight triage routines to control false positives, curating the historical corpus to sustain quality, and framing reuse gains in terms of business value, such as effort saved and defect containment.

Looking ahead, future work will build on these findings by (i) automating taxonomy-based story classification with pretrained language models (e.g., NoRBERT [24]), reducing the manual effort highlighted in this study; (ii) exploring hybrid similarity and retrieve–then–generate pipelines to improve precision while preserving recall, addressing the observed trade-offs between offline and online evaluation; (iii) extending validation to diverse domains and larger repositories, testing the generalizability of the approach beyond Web Information Systems; and (iv) conducting human-centered studies to measure cognitive load, trust, and effort savings in test triage, complementing traditional IR metrics. These directions aim to scale the approach while directly tackling the practical and research implications uncovered in our results.

Artifacts Availability

All artifacts supporting this study—including the source code of the RecSys, experimental datasets, configuration scripts, and validation results—are publicly available to foster transparency, reproducibility, and reuse by the research community[1].

Acknowledgments

This work has been partially funded by the project “*iSOP Base: Investigação e desenvolvimento de base arquitetural e tecnológica da Intelligent Sensing Operating Platform (SOP)*” supported by CENTRO DE COMPETÊNCIA EMBRAPPI VIRTUS EM HARDWARE INTELIGENTE PARA INDÚSTRIA - VIRTUS-CC, with financial resources from the PPI HardwareBR of the MCTI grant number 055/2023, signed with EMBRAPPI.

References

- [1] 2025. Supplementary Material - A kNN-Based Recommender System for Test Case Reuse in Agile Software Development. Available at Zenodo. <https://doi.org/10.5281/zenodo.16131516>. Anonymous dataset.
- [2] Muhammad Abbas, Alessio Ferrari, Anas Shatnawi, Eduard Enoiu, Mehrdad Saadatmand, and Daniel Sundmark. 2023. On the relationship between similar requirements and similar software: A case study in the railway domain. *Requirements Engineering* 28, 1 (2023), 23–47.
- [3] Gediminas Adomavicius and Alexander Tuzhilin. 2005. Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions. *IEEE Trans Knowl Data Eng* 17, 6 (2005), 734–749. arXiv:3 doi:10.1109/TKDE.2005.99
- [4] Charu C. Aggarwal, Alexander Hinneburg, and Daniel A. Keim. 2001. On the surprising behavior of distance metrics in high dimensional space. *Lect. Notes Comput. Sci.* 1973 (2001), 420–434. doi:10.1007/3-540-44503-x_27
- [5] X. Amatriain and J. M. Pujol. 2011. Data Mining Methods for Recommender Systems. In *Recommender Systems Handbook*.
- [6] Maral Azizi. 2021. A Tag-based Recommender System for Regression Test Case Prioritization. In *2021 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*. 146–157. doi:10.1109/ICSTW52544.2021.00035
- [7] Maral Azizi and Hyunsook Do. 2018. A collaborative filtering recommender system for test case prioritization in web applications. In *Proceedings of the*

- 33rd Annual ACM Symposium on Applied Computing (Pau, France) (SAC '18). Association for Computing Machinery, New York, NY, USA, 1560–1567. doi:10.1145/3167132.3167299
- [8] Bader Alshemaimri Batool Alawaji and, Mona Hakami and. 2024. Evaluating Generative Language Models with Prompt Engineering for Categorizing User Stories to its Sector Domains. In *Proceedings of the IEEE Conference*. <https://ieeexplore.ieee.org/document/10544242/>
- [9] Palash Bera and Abhimanyu Gupta. 2017. A proposed method in agile practices to create requirements documentation and test cases. In *CEUR Workshop Proceedings*, Vol. 1859. 113–121.
- [10] Pierre Bourque and Richard E. Fairley (Eds.). 2014. *SWEBOK: Guide to the Software Engineering Body of Knowledge* (version 3.0 ed.). IEEE Computer Society, Los Alamitos, CA. <http://www.swebok.org/>
- [11] Heng Tze Cheng, Levent Koc, Jeremiah Harmsen, Tal Shaked, Tushar Chandra, Hrishu Aradhye, Glen Anderson, Greg Corrado, Wei Chai, Mustafa Ispir, Rohan Anil, Zakaria Haque, Lichan Hong, Vihan Jain, Xiaobing Liu, and Hemal Shah. 2016. Wide & deep learning for recommender systems. *ACM ICPS 15-September-2016* (2016), 7–10. doi:10.1145/2988450.2988454
- [12] Geoff Cumming and Robert Calin-Jageman. 2017. *Introduction to the New Statistics: Estimation, Open Science, and Beyond*.
- [13] Emanuel Dantas, Alexandre Costa, Marcus Vinicius, Mirko Perkusich, Hyggo Almeida, and Angelo Perkusich. 2019. An Effort Estimation Support Tool for Agile Software Development: An Empirical Evaluation. 82–87. doi:10.18293/SEKE2019-141
- [14] E. Dilorenzo, E. Dantas, M. Perkusich, F. Ramos, A. Costa, D. Albuquerque, H. Almeida, and A. Perkusich. 2020. Enabling the Reuse of Software Development Assets Through a Taxonomy for User Stories. *IEEE Access* 8 (2020), 107285–107300.
- [15] Randa Elamin and Rasha Osman. 2017. Towards requirements reuse by implementing traceability in agile development. In *IEEE 41st COMPSAC*, Vol. 2. IEEE, 431–436.
- [16] B. Kantor F. Ricci, L. Rokach, B. Shapira. 2010. *Recommender System Handbook*.
- [17] Salvador García, Alberto Fernández, Julián Luengo, and Francisco Herrera. 2010. Advanced nonparametric tests for multiple comparisons in the design of experiments in computational intelligence and data mining: Experimental analysis of power. *Information Sciences* 180, 10 (2010), 2044–2064. doi:10.1016/j.ins.2009.12.010
- [18] Marko Gasparic and Andrea Janes. 2016. What recommendation systems for software engineering recommend: A systematic literature review. *J Syst Softw* 113 (2016), 101–113. doi:10.1016/j.jss.2015.11.036
- [19] Jianxin Ge and Jiaomin Liu. 2016. Software test cases recommendation system research based on collaborative filtering. 557–562. doi:10.1109/SNPD.2016.7515958
- [20] R. Gupta, Prabhulinga Manikreddy, and V. AbhinandanG. 2016. Challenges in Adapting Agile Testing in a Legacy Product. *2016 IEEE 11th ICGSE* (2016), 104–108.
- [21] Alexandru Guzu, Georgian Nicolae, Horia Cucu, and Corneliu Burileanu. 2025. Large Language Models for C Test Case Generation: A Comparative Analysis. *Electronics* 14, 11 (2025). doi:10.3390/electronics14112284
- [22] J. L. Herlocker, J. A. Konstan, and J. Riedl. 2000. Explaining collaborative filtering recommendations. *Proceedings of the ACM CSCW* (2000), 241–250. doi:10.1145/358916.358995
- [23] Tobias Hey, Dominik Fuchß, Jan Keim, and Anne Koziolk. 2025. Requirements Traceability Link Recovery via Retrieval-Augmented Generation. In *Requirements Engineering: Foundation for Software Quality*, Anne Hess and Angelo Susi (Eds.). Springer Nature Switzerland, Cham, 381–397.
- [24] Tobias Hey, Jan Keim, Anne Koziolk, and Walter F Tichy. 2020. NoBERT: Transfer learning for requirements classification. In *2020 IEEE 28th international requirements engineering conference (RE)*. IEEE, 169–179.
- [25] Rashina Hoda, Norsarema Sallah, and John Grundy. 2018. The rise and evolution of agile software development. *IEEE Software* 35, 5 (2018), 58–63.
- [26] Giuseppe Jurman, Samantha Riccadonna, Roberto Visintainer, and Cesare Furlanello. 2009. Canberra distance on ranked lists. In *NIPS Workshop*. Cite-seer, 22–27.
- [27] Brahma Reddy Korraprolu, Pavitra Pinninti, and Y. Raghu Reddy. 2025. Test Case Generation for Requirements in Natural Language - An LLM Comparison Study. In *Proceedings of the 18th Innovations in Software Engineering Conference (ISEC '25)*. Association for Computing Machinery, New York, NY, USA, Article 9, 5 pages. doi:10.1145/3717383.3717389
- [28] Mathias Landhäuser and Adrian Genaid. 2012. Connecting User Stories and code for test development. *RSE - Proceedings* (2012), 33–37. doi:10.1109/RSE.2012.6233406
- [29] Casper Lassenius, Torgeir Dingsoyr, and Maria Paasivaara. 2015. Agile processes, in software engineering, and extreme programming: 16th international conference, XP 2015 Helsinki, Finland, may 25–29, 2015 proceedings. *LNBIP* 212 (2015), 27–39. doi:10.1007/978-3-319-18612-2
- [30] Ya Jun Leng, Qing Lu, and Chang Yong Liang. 2014. Survey of recommendation based on collaborative filtering. *Moshi Shibie yu Rengong Zhineng/Pattern Recognition and Artificial Intelligence* 27, 8 (2014), 720–734. doi:10.1088/1742-6596/1314/1/012078
- [31] Jinfeng Lin, Yalin Liu, Qingkai Zeng, Meng Jiang, and Jane Cleland-Huang. 2021. Traceability transformed: Generating more accurate links with pre-trained bert models. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 324–335.
- [32] Xianchang Luo, Yinxing Xue, Zhenchang Xing, and Jiamou Sun. 2022. Prcbert: Prompt learning for requirement classification using bert-based pretrained language models. In *Proceedings of the 37th IEEE/ACM international conference on automated software engineering*. 1–13.
- [33] Normadiah Mohd Razali and Yap Bee Wah. 2011. Power comparisons of Shapiro-Wilk, Kolmogorov-Smirnov, Lilliefors and Anderson-Darling tests. *Stat Modelling* 2, 1 (2011), 21–33.
- [34] Xia Ning, Christian Desrosiers, and G. Karypis. 2015. A Comprehensive Survey of Neighborhood-Based Recommendation Methods. In *Recommender Systems Handbook*.
- [35] Felipe Ramos, Alexandre Costa, Mirko Perkusich, Hyggo Almeida, and Angelo Perkusich. 2018. A non-functional requirements recommendation system for scrum-based projects. *SEKE 2018-July, July (2018)*, 149–154. doi:10.18293/SEKE2018-107
- [36] Felipe Ramos, Alexandre Costa, Mirko Perkusich, Luiz Silva, Dalton Valadares, Ademir de Sousa Neto, Felipe Cunha, Hyggo Almeida, and Angelo Perkusich. 2025. A Data-Driven Recommendation System for Enhancing Non-Functional Requirements Elicitation in Scrum-Based Projects. *IEEE Access* 13 (2025), 44000–44023. doi:10.1109/ACCESS.2025.3548631
- [37] Felipe Ramos, Antonio Pedro, Marcos Cesar, Alexandre Costa, Mirko Perkusich, Hyggo Almeida, and Angelo Perkusich. 2019. Evaluating Software Developers' Acceptance of a Tool for Supporting Agile Non-Functional Requirement Elicitation. 26–31. doi:10.18293/SEKE2019-107
- [38] Francesco Ricci, Lior Rokach, Bracha Shapira, and Paul B. Kantor. 2010. *Recommender Systems Handbook* (1st ed.). Springer-Verlag, Berlin, Heidelberg.
- [39] Martin Robillard, Robert Walker, and Thomas Zimmermann. 2010. Recommendation Systems for Software Engineering. *IEEE Software* 27, 4 (2010), 80–86. doi:10.1109/MS.2009.161
- [40] Kenneth A Ross. 2009. Cache-Aware Query Processing Cache-Conscious Query Processing. (2009).
- [41] Günther Ruhe. 2003. Software engineering decision support: methodology and applications. *Innovations in decision support systems* 3 (2003), 143–174.
- [42] Summra Saleem, Muhammad Nabeel Asim, Ludger Van Elst, and Andreas Dengel. 2023. FNReq-Net: A hybrid computational framework for functional and non-functional requirements classification. *Journal of King Saud University-Computer and Information Sciences* 35, 8 (2023), 101665.
- [43] Andrew I. Schein, Alexandrin Popescu, Lyle H. Ungar, and David M. Pennock. 2002. Methods and metrics for cold-start recommendations. *SIGIR Forum* (2002), 253–260. doi:10.1145/564418.564421
- [44] John Slankas and Laurie Williams. 2013. Automated extraction of non-functional requirements in available documentation. In *2013 1st International workshop on natural language analysis in software engineering (NaturaliSE)*. IEEE, 9–16.
- [45] Weisong Sun, Weidong Qian, Bin Luo, and Zhenyu Chen. 2023. MuTCR: Test Case Recommendation via Multi-Level Signature Matching. In *2023 IEEE/ACM International Conference on Automation of Software Test (AST)*. 179–190. doi:10.1109/AST58925.2023.00022
- [46] Weisong Sun, Qianjun Zhang, Chunrong Fang, Yuchen Chen, Xingya Wang, and Ziyuan Wang. 2022. Test case recommendation based on balanced distance of test targets. *Information and Software Technology* 150 (2022), 106994. doi:10.1016/j.infsof.2022.106994
- [47] Diego Vergara, Antonio del Bosque, Georgios Lampropoulos, and Pablo Fernández-Arias. 2025. Trends and Applications of Artificial Intelligence in Project Management. *Electronics* 14, 4 (2025). doi:10.3390/electronics14040800
- [48] Junjie Wang, Yuchao Huang, Chunyang Chen, Zhe Liu, Song Wang, and Qing Wang. 2024. Software Testing With Large Language Models: Survey, Landscape, and Vision. *IEEE Transactions on Software Engineering* 50, 4 (2024), 911–936. doi:10.1109/TSE.2024.3368208
- [49] Xinyu Wang, Liping Zhao, Ye Wang, and Jie Sun. 2014. The role of requirements engineering practices in agile development: an empirical study. In *Requirements Engineering*. Springer, 195–209.
- [50] Moshi Wei, Nima Shiri Harzevili, Yuchao Huang, Junjie Wang, and Song Wang. 2022. Clear: contrastive learning for api recommendation. In *Proceedings of the 44th International Conference on Software Engineering*. 376–387.
- [51] Claes Wohlin, Per Runeson, Martin Hst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslin. 2012. *Experimentation in Software Engineering*. Springer Publishing Company, Incorporated.
- [52] Wansheng Yang, Fei Deng, Siyou Ma, Linbo Wu, Zhe Sun, and Chi Hu. 2021. Test Case Reuse Based on Software Testing Knowledge Graph and Collaborative Filtering Recommendation Algorithm. In *2021 IEEE 21st International Conference on Software Quality, Reliability and Security Companion (QRS-C)*. 67–76. doi:10.1109/QRS-C5045.2021.00020
- [53] Jianfei Zhu, Guanping Xiao, Zheng Zheng, and Yulei Sui. 2024. Deep semi-supervised learning for recovering traceability links between issues and commits. *Journal of Systems and Software* 216 (2024), 112109. doi:10.1016/j.jss.2024.112109